27

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

예외 필터와 예외 계층

제2부 NestJS 핵심 · 6장 요청 파이프라인

던진 실패가 HTTP 응답이 되는 길과, 그 길을 벗어나는 실패



© 한상훈 NestJS 백엔드 마스터

500이 된 매진 좌석

```
// 가정: 좌석 서비스가 일반 Error를 던진 코드
throw new Error(`${seatLabel(seat.zone, seat.row, seat.no)}은 이미 팔렸거나 선택된 좌석입니다`);
```

○ ○ ○ curl · 서버 로그

```
$ curl -X POST localhost:4027/performances/1/seats/A-1-1/hold   (두 번째)
{"type":"about:blank","title":"Internal Server Error","status":500,
 "detail":"요청을 처리하지 못했습니다.", …} [500]

[Timing] 49a7d856-… SeatsController.hold 실패(Error) 0.3ms
```

→ 에러 로그: 원래 메시지(이미 팔렸거나 선택된 좌석) 기록, 응답에는 없음

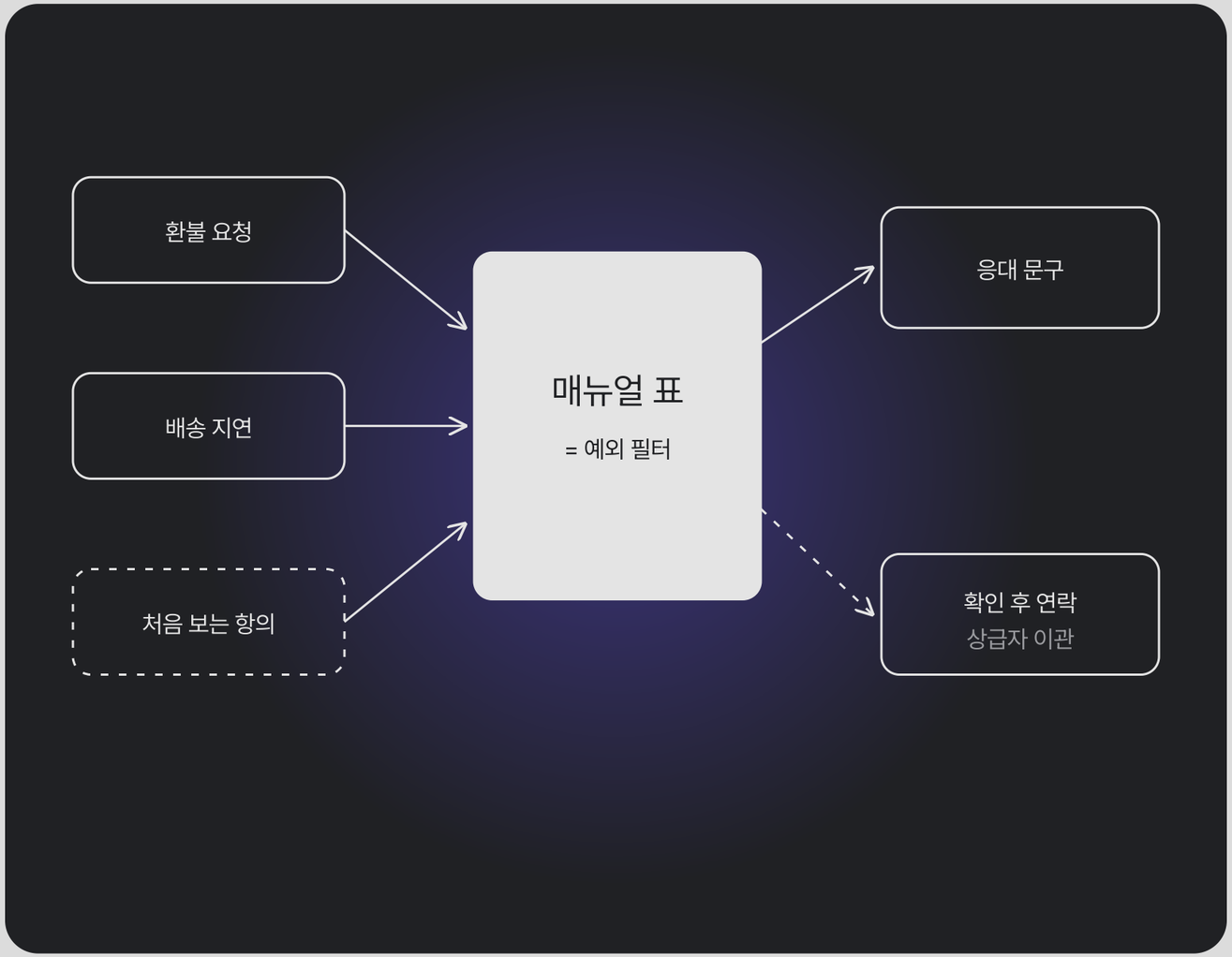
© 한상훈 NestJS 백엔드 마스터

민원 대응 매뉴얼

→ 항의 종류마다 응대 문구와 이관 절차가 정해져 있음

→ 상담원: 즉석에서 문구를 만들지 않음, 항의 종류만 판단

→ 매뉴얼에 없는 항의: 상급자에게 넘기고 "확인 후 연락" 안내



© 한상훈 NestJS 백엔드 마스터

사람이 읽을 문장, 프로그램이 읽을 type

"에러 본문에는 사람이 읽을 문장과 프로그램이 읽을 type을 따로 싣습니다."

→ 500: 서버가 모르는 실패, 클라이언트는 재시도 외에 할 일이 없음

→ 이미 팔린 좌석: 서버가 아는 실패, 409로 "다른 좌석 고르기"와 좌석 지도 새로 불러오기

© 한상훈 NestJS 백엔드 마스터

HttpException

```
new HttpException('공연 9번이 없습니다', HttpStatus.NOT_FOUND);
new HttpException({ message: '…', reason: 'sold-out' }, 409);
new ConflictException('이미 팔린 좌석', { cause: err });   // 원래 에러는 cause로
```

→ 첫 인자: 응답 본문의 재료 (문자열 또는 객체)

→ 둘째 인자: 상태 코드

→ 옵션 cause: 원인 에러를 함께 보관 (응답에는 나가지 않음)

© 한상훈 NestJS 백엔드 마스터

내장 예외 클래스

클래스	상태 코드	이 프로젝트에서
`BadRequestException`	400	ParseIntPipe, ValidationPipe, SeatIdPipe
`UnauthorizedException`	401	직원 키 없음
`ForbiddenException`	403	기획팀 직원, false를 돌려준 가드
`NotFoundException`	404	없는 공연, 없는 회차
`ConflictException`	409	이미 팔린 좌석
`UnprocessableEntityException`	422	(아직 없음)
`ServiceUnavailableException`	503	점검 중 미들웨어
`GatewayTimeoutException`	504	타임아웃 인터셉터

→ 전부 HttpException의 하위 클래스

→ 422 예: 모양은 맞지만 규칙상 불가 (공연 시작 10분 전 예매)

© 한상훈 NestJS 백엔드 마스터

getResponse()의 세 모양

던진 것	getResponse()
`new NotFoundException('공연 9번이 없습니다')`	`{ message: '공연 9번이 없습니다', error: 'Not Found', statusCode: 404 }`
ValidationPipe 실패	`{ message: [ '…', '…' ], error: 'Bad Request', statusCode: 400 }`
`new HttpException('문자열', 409)`	`'문자열'`

→ 필터는 세 모양을 모두 처리해야 함

© 한상훈 NestJS 백엔드 마스터

지금까지 만난 상태 코드

실패	던진 것	응답
이미 팔린 좌석	`Error`	500
이미 팔린 좌석	`ConflictException`	409
264KB 본문	`PayloadTooLargeError` (http-errors)	500

→ HttpException이 아닌 것: 전부 500 (지금 필터 기준)

© 한상훈 NestJS 백엔드 마스터

예외 필터

→ 던져진 예외를 받아 HTTP 응답을 직접 만드는 클래스

→ `ExceptionFilter` 구현, `catch(exception, host)` 하나

→ `@Catch(종류…)`: 받을 예외 클래스, 비우면 모든 예외

→ `host`: ArgumentsHost (실행 컨텍스트의 부모, getClass·getHandler 없음)

→ 필터가 응답을 보내지 않으면 요청이 끝나지 않음

© 한상훈 NestJS 백엔드 마스터

ProblemDetailsFilter의 세 갈래

problem-details.filter.ts

```
@Catch()
export class ProblemDetailsFilter implements ExceptionFilter {
  catch(exception: unknown, host: ArgumentsHost) {
    // …
    if (exception instanceof ProblemException) {
      // type, title, 확장 필드를 예외가 직접 가짐
    } else if (exception instanceof HttpException) {
      // 상태 코드는 getStatus(), message가 배열이면 errors, 문자열이면 detail
    } else {
      // 내부 사정: 로그에만, 응답은 500 "요청을 처리하지 못했습니다."
    }
    body.instance = req.originalUrl;
    res.status(status).type('application/problem+json').json(body);
  }
}
```

© 한상훈 NestJS 백엔드 마스터

413 되찾기

```
} else if (isClientHttpError(exception)) {
  // Express 미들웨어(body parser 등)가 던진 4xx 에러: 상태 코드 그대로 사용
  status = exception.status;
  body = { type: 'about:blank', title: STATUS_CODES[status], status, detail: exception.message };
}

// http-errors 모양(status 숫자 + expose)의 4xx 에러인가
function isClientHttpError(e: unknown): e is Error & { status: number } {
  const s = (e as { status?: unknown })?.status;
  return e instanceof Error && typeof s === 'number' && s >= 400 && s < 500
    && (e as { expose?: boolean }).expose === true;
}
```

○ ○ ○ curl

```
$ curl -X POST localhost:4027/concerts --data-binary @layout.json
{"type":"about:blank","title":"Payload Too Large","status":413,
 "detail":"request entity too large","instance":"/concerts"} [413]
```

© 한상훈 NestJS 백엔드 마스터

HTTP를 아는 서비스

- → 지금의 좌석 서비스: `ConflictException`, `NotFoundException`을 직접 던짐
- → 가정: 알림 워커나 정산 배치가 같은 서비스를 호출
- → 그곳의 실패: HTTP 응답이 없는데 "Conflict" 예외
- → 서비스가 알아야 할 것: 무엇이 실패했는가
- → 필터가 정할 것: 어떤 상태 코드와 본문으로 보낼까

© 한상훈 NestJS 백엔드 마스터

DomainError와 좌석 에러

domain-error.ts

```ts
// 업무 규칙 실패 예외, HTTP를 모름
export type DomainErrorKind = 'not-found' | 'conflict' | 'rejected';

export abstract class DomainError extends Error {
  abstract readonly kind: DomainErrorKind;
  abstract readonly code: string;

  constructor(message: string, readonly details: Record<string, unknown> = {}) {
    super(message);
    this.name = new.target.name;
  }
}
```

seat.errors.ts

```ts
export class SeatNotFoundError extends DomainError {
  readonly kind = 'not-found';
  readonly code = 'seat-not-found';
}
export class SeatUnavailableError extends DomainError {
  readonly kind = 'conflict';
  readonly code = 'seat-unavailable';
}
```

© 한상훈 NestJS 백엔드 마스터

DomainErrorFilter

domain-error.filter.ts

```ts
const STATUS: Record<DomainErrorKind, number> = { 'not-found': 404, conflict: 409, rejected: 422 };

@Catch(DomainError)
export class DomainErrorFilter implements ExceptionFilter {
  private readonly logger = new Logger('DomainError');

  catch(error: DomainError, host: ArgumentsHost) {
    const req = host.switchToHttp().getRequest<Request>();
    const res = host.switchToHttp().getResponse<Response>();
    const status = STATUS[error.kind];
    this.logger.warn(`${req.headers['x-request-id']} ${error.name}: ${error.message}`);
    res.status(status).type('application/problem+json').json({
      ...error.details, // 확장 필드가 표준 칸을 덮지 못하게 먼저
      type: `https://api.ticketing.example/problems/${error.code}`,
      title: error.name, status, detail: error.message, instance: req.originalUrl,
    });
  }
}
```

© 한상훈 NestJS 백엔드 마스터

겹친 status

curl

```
(확장 필드를 뒤에 펼친 코드, 세부 정보에 status: seat.status)
{"type":"https://api.ticketing.example/problems/seat-unavailable",
 "title":"SeatUnavailableError","status":"HELD", …} [409]

(고친 코드: 확장 필드를 먼저 펼치고 이름을 seatStatus로)
{"seatId":"A-1-1","seatStatus":"HELD",
 "type":"https://api.ticketing.example/problems/seat-unavailable",
 "title":"SeatUnavailableError","status":409,
 "detail":"A구역 1열 1번은 이미 팔렸거나 선택된 좌석입니다", …} [409]

{"seatId":"A-9-9","type":"https://api.ticketing.example/problems/seat-not-found",
 "title":"SeatNotFoundError","status":404,"detail":"좌석 A-9-9이 없습니다", …} [404]
```

→ 상태 줄은 409, 본문 status는 `"HELD"`: 본문을 읽는 클라이언트가 깨짐

→ type으로 다음 행동: `seat-unavailable` → 좌석 지도 다시, `seat-not-found` → 회차 화면으로

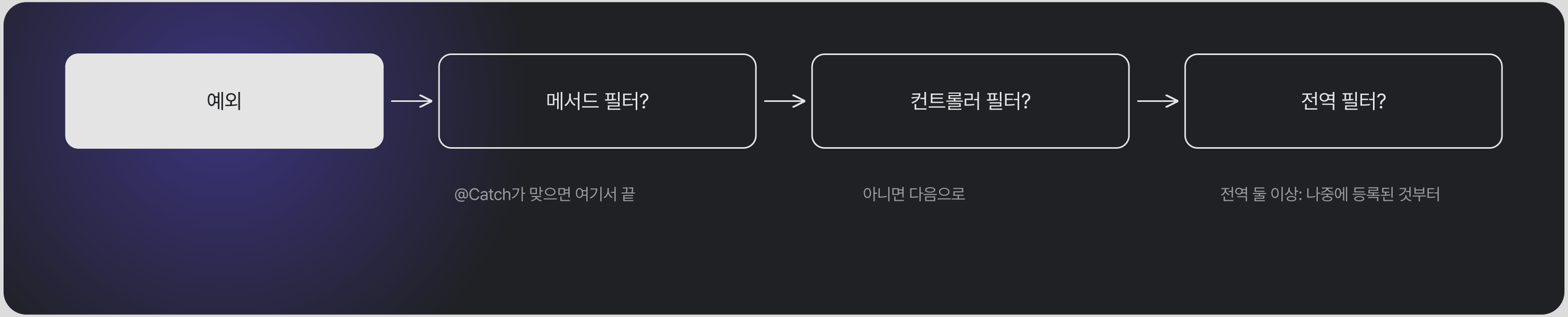
© 한상훈 NestJS 백엔드 마스터

필터를 고르는 순서

→ 거는 곳: 메서드(`@UseFilters`), 컨트롤러(`@UseFilters`), 전역

→ 고르는 순서: 메서드 → 컨트롤러 → 전역 (가드·인터셉터와 반대)

→ `@Catch`가 맞는 필터 하나만 실행, 그다음 필터로 넘기지 않음



© 한상훈 NestJS 백엔드 마스터

전역 필터 두 개

등록	이미 팔린 좌석
APP_FILTER(DomainErrorFilter) + main.ts useGlobalFilters(ProblemDetailsFilter)	500
useGlobalFilters(ProblemDetailsFilter, DomainErrorFilter)	409
useGlobalFilters(DomainErrorFilter, ProblemDetailsFilter)	500
APP_FILTER [ProblemDetailsFilter, DomainErrorFilter]	409
APP_FILTER [DomainErrorFilter, ProblemDetailsFilter]	500

→ 나중에 등록된 전역 필터가 먼저 고름

→ @Catch()가 빈 필터가 먼저 고르면 나머지는 기회가 없음

데이터 블록: @nestjs/core 12.2.0, 등록 다섯 가지 실측 (2026-10-26), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

두 필터를 한 곳에

app.module.ts

```
providers: [
  AppService,
  { provide: APP_INTERCEPTOR, useClass: TimingInterceptor },
  // 전역 필터: 나중에 적은 것이 먼저 고른다. 모든 예외를 받는 ProblemDetailsFilter를 맨 앞에
  { provide: APP_FILTER, useClass: ProblemDetailsFilter },
  { provide: APP_FILTER, useClass: DomainErrorFilter },
],
```

→ main.ts의 `useGlobalFilters` 삭제

→ 둘 다 Nest가 생성: 주입 가능

© 한상훈 NestJS 백엔드 마스터

매표소만 다른 에러 모양

```
// 매표소 단말기 옛 버전이 읽던 에러 모양 (가정)
@Catch(HttpException)
export class LegacyHttpFilter implements ExceptionFilter { /* { code, msg } */ }

@UseGuards(StaffKeyGuard)
@UseFilters(LegacyHttpFilter)
export class BoxOfficeController { … }
```

```
○ ○ ○  curl
─────────────────────────────────────────────────────────────────────────────
키 없음       {"code":401,"msg":"직원 키가 없거나 맞지 않습니다"} [401]
좌석 형식     {"code":400,"msg":"seatId 형식이 아닙니다: A15 (예: A-3-15)"} [400]
이미 팔림     {"seatId":"A-1-1","seatStatus":"SOLD","type":"…/seat-unavailable", …} [409]
```

→ 한 컨트롤러 안에서도 모양이 둘

© 한상훈 NestJS 백엔드 마스터

기다리지 않은 감사 기록

audit.service.ts

```ts
// 예매 감사 기록 저장 (외부 저장소 흉내). AUDIT_FAIL=on이면 실패
async write(event: string) {
  await new Promise((r) => setTimeout(r, 100));
  if (process.env.AUDIT_FAIL === 'on') throw new Error(`감사 저장소 응답 없음: ${event}`);
}
```

```ts
// ReservationsService.reserve
this.logger.log(`${seatId} 결제 완료`);
void this.audit.write(`reserve ${seatId}`); // 기다리지 않고 던져 둠
```

© 한상훈 NestJS 백엔드 마스터

201 뒤에 꺼진 서버

○ ○ ○ curl · 서버 로그

```
$ curl -X POST localhost:4027/reservations -d '{"seatId":"A-1-1","amount":110000,…}'
{"id":1,"seatId":"A-1-1","approvalId":"fake-110000", …} [201]

[예매] A-1-1 결제 완료
LOG [HTTP] a85ecc8a-… POST /reservations 201 2.5ms
Error: 감사 저장소 응답 없음: reserve A-1-1
Node.js v24.11.0
(서버 프로세스 종료 코드 1)

$ curl -m 2 localhost:4027/concerts
 [000]
```

© 한상훈 NestJS 백엔드 마스터

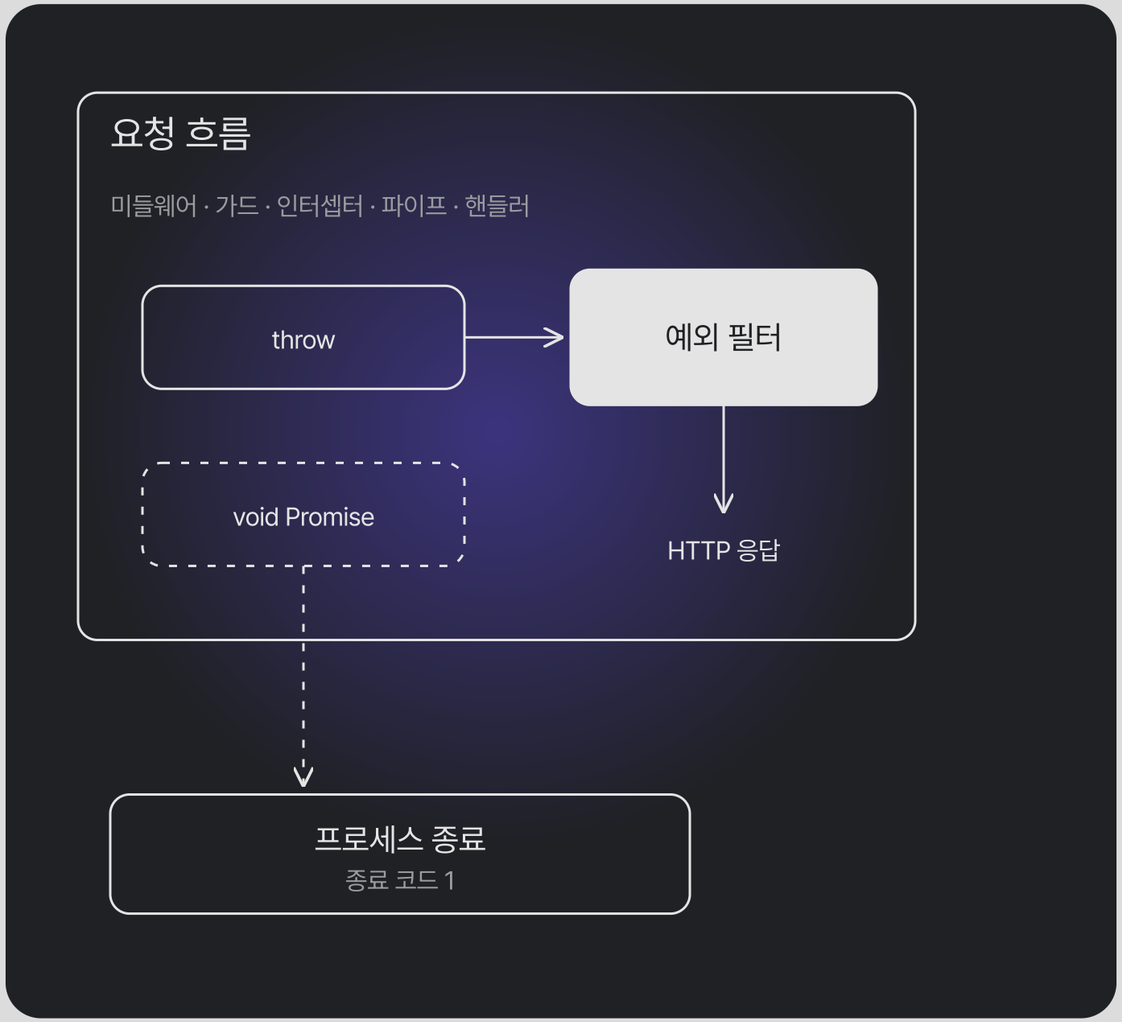
필터가 받는 범위

→ 받음: 미들웨어, 가드, 인터셉터, 파이프, 핸들러에서 던진 예외, 핸들러가 돌려준 Promise의 실패

→ 못 받음: 응답과 상관없이 따로 돌린 Promise의 실패 (void, await 없음)

→ 못 받음: `setTimeout`·이벤트 콜백 안에서 던진 예외

→ 처리 안 된 Promise 실패: Node가 프로세스를 종료 (기본 동작)



데이터 블록: Node v24.11.0 실측: 처리 안 된 Promise 실패 → 종료 코드 1 (2026-10-26), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

.catch()로 받기

```
this.audit.write(`reserve ${seatId}`).catch((err) => this.logger.log(`감사 기록 실패: ${err.message}`));
```

○ ○ ○ curl · 서버 로그

```
{"id":1,"seatId":"A-1-1", …} [201]
[예매] A-1-1 결제 완료
[예매] 감사 기록 실패: 감사 저장소 응답 없음: reserve A-1-1
$ curl localhost:4027/concerts
[{"id":1,"title":"가을 콘서트","venue":"올림픽홀"}] [200]
```

→ 기다리지 않을 작업: 실패를 받을 곳을 직접 붙임

© 한상훈 NestJS 백엔드 마스터

실패가 응답이 되는 길

던진 것	받는 필터	응답
DomainError (SeatUnavailableError 등)	`DomainErrorFilter`	404·409·422, type URL
ProblemException	`ProblemDetailsFilter`	예외가 가진 type
HttpException 하위 클래스	`ProblemDetailsFilter`	그 상태 코드, about:blank
4xx http-errors (body parser)	`ProblemDetailsFilter`	413 등
그 밖의 Error	`ProblemDetailsFilter`	500, 내부 사정은 로그만
요청 흐름 밖의 실패	없음	프로세스 종료 → .catch로 직접

© 한상훈 NestJS 백엔드 마스터

예외 필터

"서비스는 무엇이 실패했는지만 던지고, 그걸 어떤 응답으로 바꿀지는 예외 필터가 정합니다."

"예외 필터는 요청을 처리하는 흐름 안에서 던진 예외만 받습니다."